

NYC Taxi Trips Analysis

January – December 2023

Prepared by: Roma Agrawal

New York City Yellow Taxi Data

Objective

As an analyst at a growing taxi operation in New York City, I analyzed the 2023 yellow taxi trip data to uncover actionable insights that can help optimize taxi services. The primary objective was to identify patterns and trends that can inform strategic decisions aimed at improving operational efficiency, maximizing revenue, and enhancing the overall passenger experience.

Business Value

Taxis are a vital component of NYC's dynamic urban transport network. With constantly shifting demand patterns, it is essential for taxi operators to adapt and optimize operations. This report uses Exploratory Data Analysis (EDA) to provide actionable insights that support operational improvements, drive profitability, and improve customer satisfaction.

Data Understanding

The yellow taxi trip records include fields capturing pick-up and drop-off dates/times, pick-up and drop-off locations, trip distances, itemized fares, rate types, payment types, and driver-reported passenger counts.

The data is stored in Parquet format (*.parquet*). The dataset is from 2009 to 2024. However, for this assignment, we will only be using the data from 2023.

The data for each month is present in a different parquet file. You will get twelve files for each of the months in 2023.

The data was collected and provided to the NYC Taxi and Limousine Commission (TLC) by technology providers like vendors and taxi hailing apps.

You can find the link to the TLC trip records page here:
<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

Data Description

You can find the data description here: [Data Dictionary](#)

Trip Records

Field Name	description
VendorID	A code indicating the TPEP provider that provided the record. 1= Creative Mobile Technologies, LLC; 2= VeriFone Inc.
tpep_pickup_datetime	The date and time when the meter was engaged.
tpep_dropoff_datetime	The date and time when the meter was disengaged.
Passenger_count	The number of passengers in the vehicle. This is a driver-entered value.
Trip_distance	The elapsed trip distance in miles reported by the taximeter.
PULocationID	TLC Taxi Zone in which the taximeter was engaged
DOLocationID	TLC Taxi Zone in which the taximeter was disengaged
RateCodeID	The final rate code in effect at the end of the trip. 1 = Standard rate 2 = JFK 3 = Newark 4 = Nassau or Westchester 5 = Negotiated fare 6 = Group ride
Store_and_fwd_flag	This flag indicates whether the trip record was held in vehicle memory before sending to the vendor, aka "store and forward," because the vehicle did not have a connection to the server. Y= store and forward trip N= not a store and forward trip

Payment_type	A numeric code signifying how the passenger paid for the trip. 1 = Credit card 2 = Cash 3 = No charge 4 = Dispute 5 = Unknown 6 = Voided trip
Fare_amount	The time-and-distance fare calculated by the meter. Extra Miscellaneous extras and surcharges. Currently, this only includes the 0.50 and 1 USD rush hour and overnight charges.
MTA_tax	0.50 USD MTA tax that is automatically triggered based on the metered rate in use.
Improvement_surcharge	0.30 USD improvement surcharge assessed trips at the flag drop. The improvement surcharge began being levied in 2015.
Tip_amount	Tip amount – This field is automatically populated for credit card tips. Cash tips are not included.
Tolls_amount	Total amount of all tolls paid in trip.
total_amount	The total amount charged to passengers. Does not include cash tips.
Congestion_Surcharge	Total amount collected in trip for NYS congestion surcharge.
Airport_fee	1.25 USD for pick up only at LaGuardia and John F. Kennedy Airports

Although the amounts of extra charges and taxes applied are specified in the data dictionary, you will see that some cases have different values of these charges in the actual data.

Taxi Zones

Each of the trip records contains a field corresponding to the location of the pickup or drop-off of the trip, populated by numbers ranging from 1-263.

These numbers correspond to taxi zones, which may be downloaded as a table or map/shapefile and matched to the trip records using a join.

Assumptions Made

- Only yellow taxis were analyzed.
- Trip records with zero/negative values for fare, distance, or duration were removed.
- Time periods were treated in 24-hour format.
- Pickup and drop-off zones were merged using PULocationID and DOLocationID from the NYC TLC shapefile.
- All surcharges were assumed to be applied per trip.
- Sampling fraction was used to scale trip volume and revenue estimates where relevant.

1 Data Preparation

There are total 12 parquet files one for each month. First task is to combine all the 12 files and read it.

```
df1 = pd.read_parquet("NYCC/yellow_tripdata_2023-01.parquet")
df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3066766 entries, 0 to 3066765
Data columns (total 19 columns):
#   Column              Dtype
---  -
0   VendorID            int64
1   tpep_pickup_datetime  datetime64[us]
2   tpep_dropoff_datetime datetime64[us]
3   passenger_count      float64
4   trip_distance        float64
5   RatecodeID          float64
6   store_and_fwd_flag   object
7   PULocationID         int64
8   DOLocationID         int64
9   payment_type         int64
10  fare_amount          float64
11  extra                float64
12  mta_tax              float64
13  tip_amount           float64
14  tolls_amount         float64
15  improvement_surcharge float64
16  total_amount         float64
17  congestion_surcharge float64
18  airport_fee          float64
```

```
dtypes: datetime64[us](2), float64(12), int64(4), object(1)
memory usage: 444.6+ MB
```

There were total 3066766 rows and its huge file so we did a sampling of data and did the analysis on this sampled data.

```
print("Sampled data shape:", sampled_data.shape)
# Store the df in csv/parquet
#df.to_parquet('NYCC')
Sampled data shape: (383116, 20)
```

2 Data Cleaning

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 383116 entries, 0 to 383115
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   VendorID              383116 non-null  int64
1   tpep_pickup_datetime  383116 non-null  object
2   tpep_dropoff_datetime 383116 non-null  object
3   passenger_count       370219 non-null  float64
4   trip_distance         383116 non-null  float64
5   RatecodeID           370219 non-null  float64
6   store_and_fwd_flag    370219 non-null  object
7   PULocationID          383116 non-null  int64
8   DOLocationID          383116 non-null  int64
9   payment_type          383116 non-null  int64
10  fare_amount           383116 non-null  float64
11  extra                 383116 non-null  float64
12  mta_tax               383116 non-null  float64
13  tip_amount            383116 non-null  float64
14  tolls_amount          383116 non-null  float64
15  improvement_surcharge 383116 non-null  float64
16  total_amount          383116 non-null  float64
17  congestion_surcharge  370219 non-null  float64
18  Airport_fee           340242 non-null  float64
19  airport_fee           29977 non-null   float64
dtypes: float64(13), int64(4), object(3)
```

2.1 Fixing Columns

We fixed the index and dropped unnecessary columns. We also merged in such a way that all required information moved to 'Airport_fee' and 'airport_fee' dropped.

Next we fixed the negative values on various columns.

Sample rows with negative fare amounts:

	tpep_pickup_datetime	tpep_dropoff_datetime	RatecodeID	fare_amount
5	2023-06-01 00:26:05	2023-06-01 00:26:21	1.0	-3.0
47	2023-06-01 05:53:58	2023-06-01 06:32:47	1.0	-73.0
49	2023-06-01 05:58:23	2023-06-01 05:58:49	1.0	-3.0
95	2023-06-01 07:18:27	2023-06-01 07:19:21	1.0	-3.0
102	2023-06-01 07:50:46	2023-06-01 08:09:15	1.0	-18.4

All negative 'fare_amount' values have been converted to positive.

Number of negative fare_amount values: 0

We still have some other negative values:

Column 'extra' has 1923 negative values.

Column 'mta_tax' has 3663 negative values.

Column 'tip_amount' has 17 negative values.

Column 'tolls_amount' has 243 negative values.

Column 'improvement_surcharge' has 3771 negative values.

Column 'total_amount' has 3774 negative values.

Column 'congestion_surcharge' has 3037 negative values.

Column 'Airport_fee' has 397 negative values.

Which has been fixed.

2.2 Handling Missing Values

There were proportion of missing values in each column below (%):

```
Airport_fee      10.48
congestion_surcharge  3.37
passenger_count   3.37
RatecodeID        3.37
store_and_fwd_flag  3.37
dtype: float64
```

We first handled missing values in passenger_count, there were rows with passenger count zero which we handled replacing with mode.

Replaced 'passenger_count' == 0 with mode value: 1.0

We also fixed the missing value on RatecodeID

Filled missing 'RatecodeID' with mode value: 1.0

There were Nan in congestion_surcharge which were fixed with mode value: 2.5

As of now proportion of missing values in each column (%):

```
Airport_fee      10.48
store_and_fwd_flag  3.37
dtype: float64
```

After this, I fixed the missing values in Airport_fee and store_and_fwd_flag with 0.00 and 'N' respectively

2.3 Handling Outliers

First we performed the outlier analysis

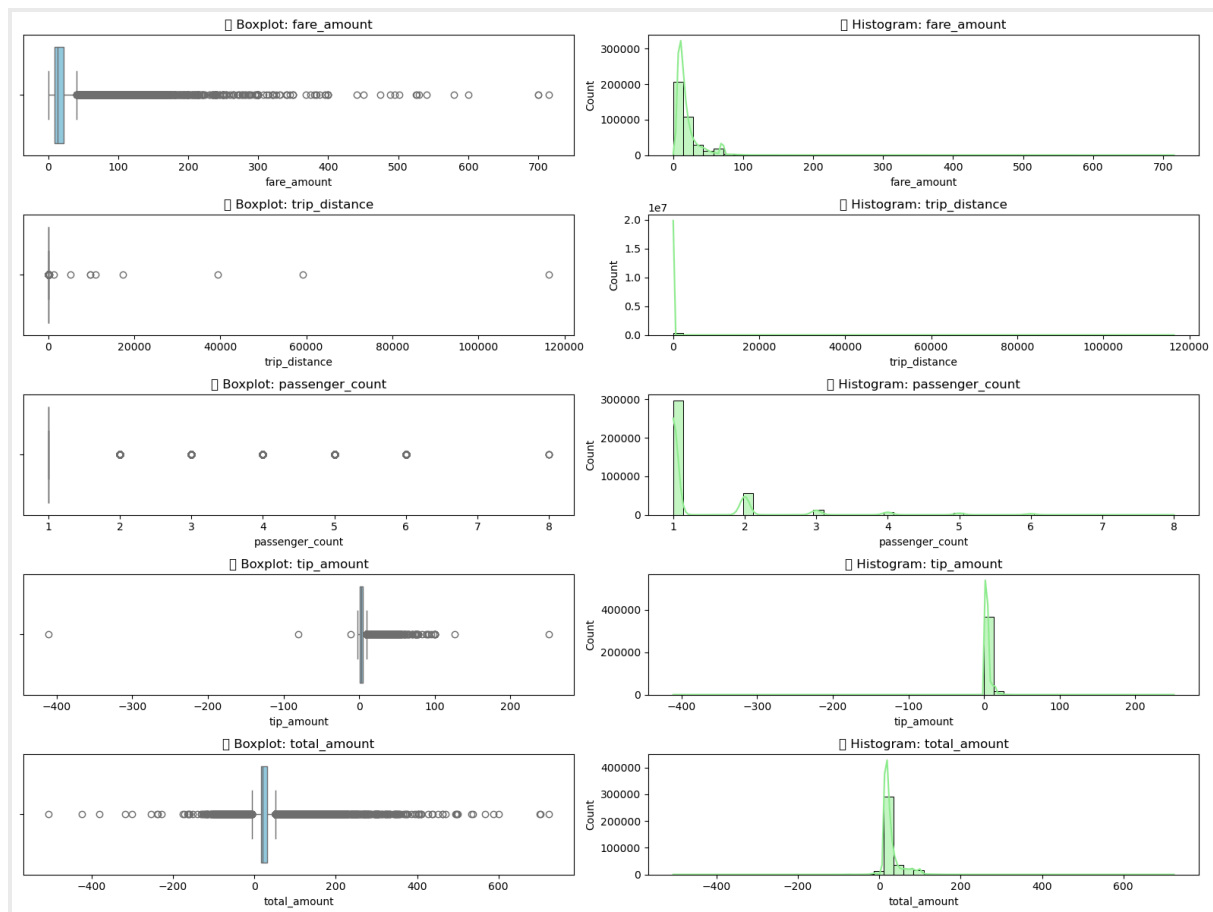
40235 potential outliers in 'fare_amount' using IQR method.

50225 potential outliers in 'trip_distance' using IQR method.

86200 potential outliers in 'passenger_count' using IQR method.

29339 potential outliers in 'tip_amount' using IQR method.

47936 potential outliers in 'total_amount' using IQR method.

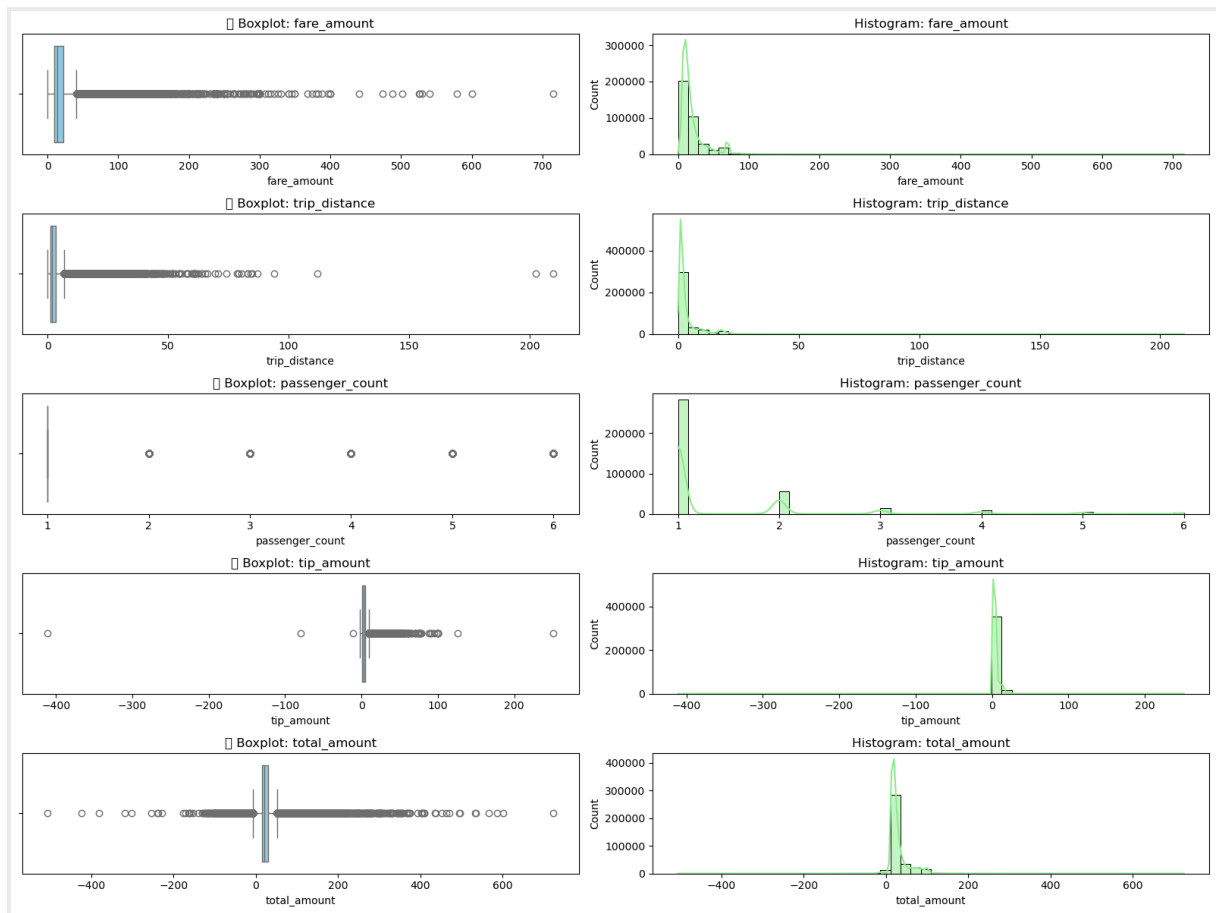


Based on the above analysis, iwe found that some of the outliers are present due to errors in registering the trips.

First, I remove 7+ passenger counts as there are very less instances.

1. Removed 4 rows with passenger_count > 6
2. Updated DataFrame now excludes passenger_count > 6
3. Removed 12 rows with trip_distance ≈ 0 and fare > 300
4. Removed 17 rows with zero fare & distance but different zones
5. Removed 10 rows with trip_distance > 250 miles
6. Removed 12891 rows with payment_type = 0 (invalid)

Now lets check again:



3 Exploratory Data Analysis

The first part of EDA is to find Patterns and Trends

Temporal Analysis

These represent quantities and can be measured, counted, or used in mathematical calculations.

- passenger_count
- trip_distance
- pickup_hour
- trip_duration

Monetary Columns (also Numerical):

These are continuous numeric values representing amounts.

- fare_amount
- extra
- mta_tax
- tip_amount
- tolls_amount
- improvement_surcharge
- total_amount
- congestion_surcharge
- airport_fee

Categorical Variables

These represent groups, labels, or codes with finite possible values.

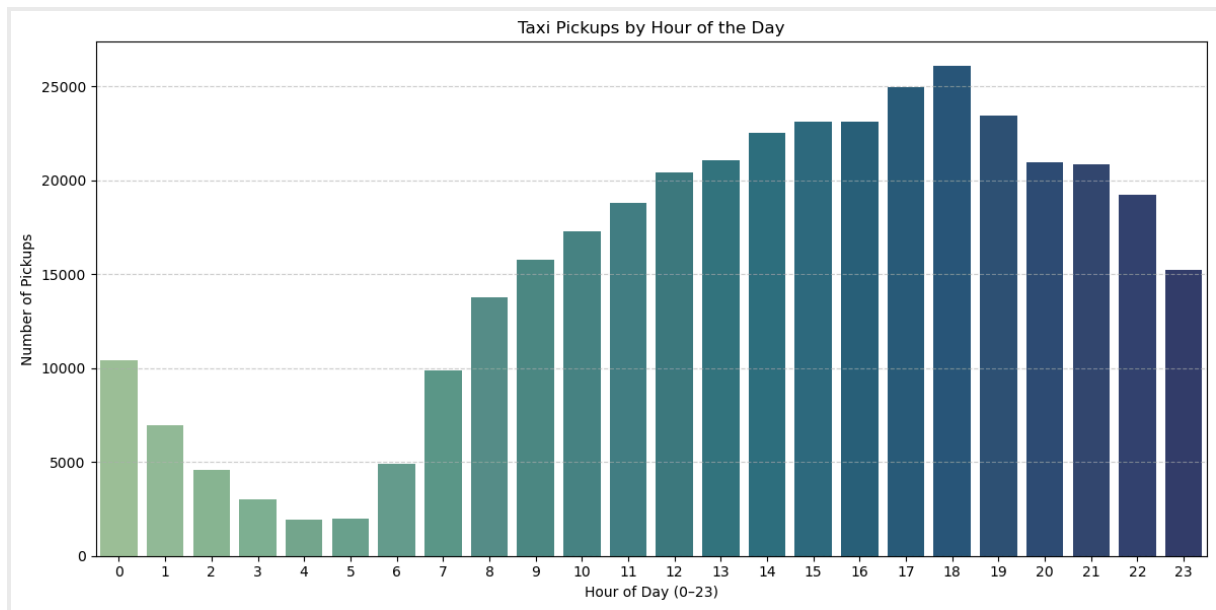
- VendorID
- tpep_pickup_datetime (*treated as datetime, not used as numerical*)
- tpep_dropoff_datetime (*same as above*)
- RatecodeID
- PULocationID
- DOLocationID
- payment_type

.

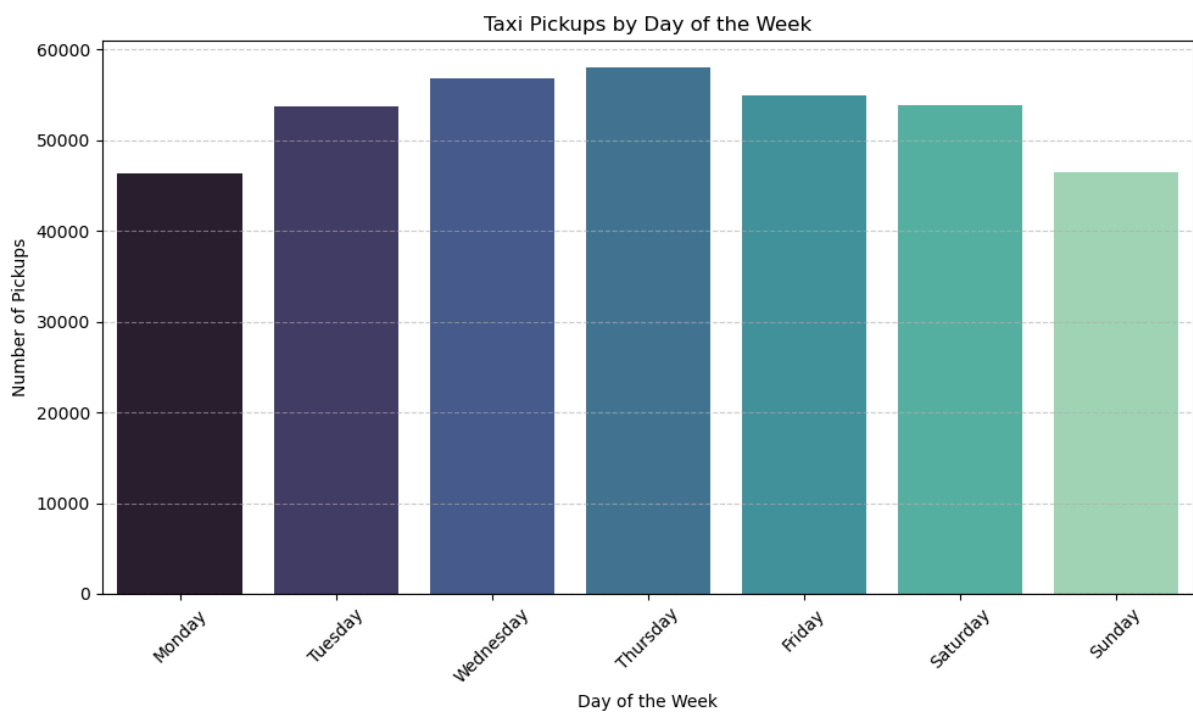
Now its time for some fun:

Let's begin with hourly trends in taxi pickups — this will help us understand demand patterns throughout the day.

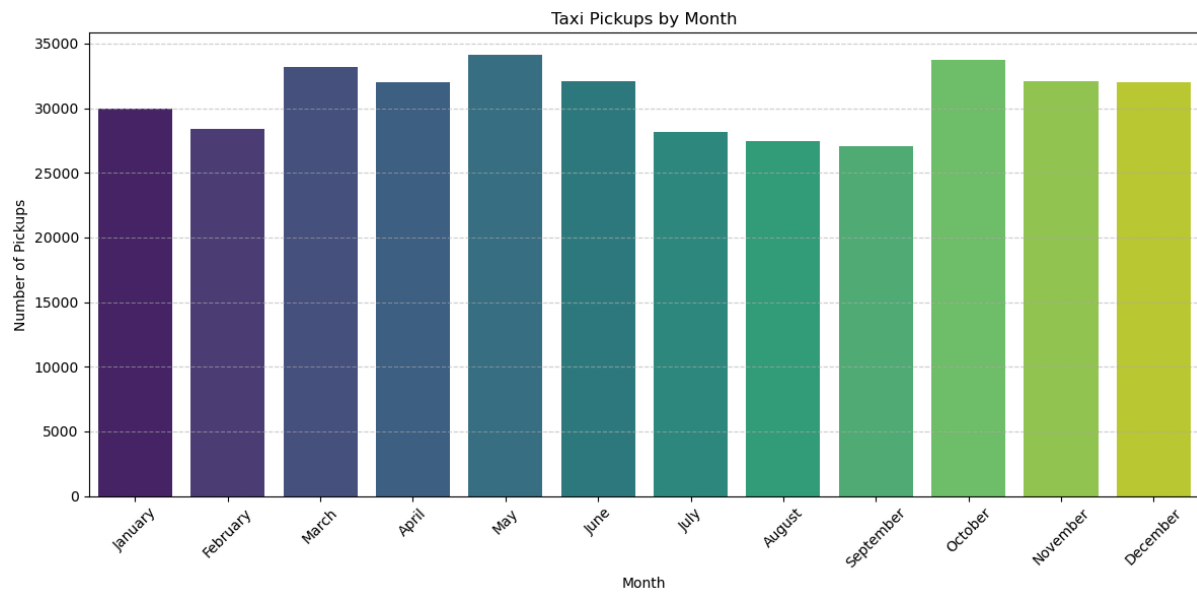
We'll use the pickup_hour column for this (or extract it from tpep_pickup_datetime if not available).



Let's now analyze the **daily trends in taxi pickups**, broken down by the **day of the week** (Monday to Sunday).



Let's now look at the **monthly trends in taxi pickups** — this helps identify seasonal or month-over-month fluctuations in demand.



Now lets do Financial Analysis

In Financial Analysis, we will investigate the values in key fare-related and distance-related columns. We'll analyze whether they contain any suspicious zero or negative values, which could indicate data issues or unusual trip behavior.

There are financial parameters like `fare_amount`, `tip_amount`, `total_amount`, and also `trip_distance`.

Lets look at the data first:

fare_amount — Total: 370182

Zero values: 120 (0.03%)

Negative values: 0 (0.00%)

tip_amount — Total: 370182

Zero values: 85711 (23.15%)

Negative values: 17 (0.00%)

total_amount — Total: 370182

Zero values: 53 (0.01%)

Negative values: 3770 (1.02%)

trip_distance — Total: 370182

Zero values: 5003 (1.35%)

Negative values: 0 (0.00%)

zero distance could be legitimate in cases like:

- Pickup & dropoff at the same location (e.g., waiting charges),
- Very short trips (under 0.1 mile) possibly rounded to 0,

- Errors in data capture (e.g., GPS glitches).

For now, we'll retain those 0-distance trips but still filter out all zero values from the key financial columns (fare_amount, tip_amount, total_amount) and keep trip_distance as-is.

We've removed suspicious records where financial parameters were exactly zero.

Now let's analyze monthly revenue trend, for this, we'll calculate the sum of total_amount grouped by each month. Since we already extracted pickup_month during earlier preprocessing, we'll use that for grouping. Below is total revenue earned from taxi trips for each month.

Monthly Revenue Summary:

	pickup_month	total_revenue	avg_revenue_per_trip	trip_count
0	2023-01	653150.48	28.357161	23033
1	2023-02	621483.83	28.099825	22117
2	2023-03	742225.15	28.964884	25625
3	2023-04	728592.63	29.502455	24696
4	2023-05	789681.51	30.027055	26299
5	2023-06	745496.24	30.091880	24774
6	2023-07	626131.50	29.825728	20993
7	2023-08	608902.69	29.792675	20438
8	2023-09	644128.39	30.987078	20787
9	2023-10	809221.43	30.794635	26278
10	2023-11	748394.84	29.895136	25034
11	2023-12	729521.05	29.910662	24390

Output Includes:

- pickup_month: Month of trip
- total_revenue: Total revenue generated that month
- avg_revenue_per_trip: Mean revenue per trip
- trip_count: Number of trips for that month

Let's break down the revenue by quarter and show each quarter's proportion of the total annual revenue.

We'll convert the pickup datetime to a quarter format (e.g., 2023Q1, 2023Q2, etc.) and then calculate the revenue contribution of each.

Revenue Proportion by Quarter:

	pickup_quarter	total_revenue	revenue_proportion
0	2023Q1	2016859.46	0.2388
1	2023Q2	2263770.38	0.2680

2	2023Q3	1879162.58	0.2225
3	2023Q4	2287137.32	0.2708

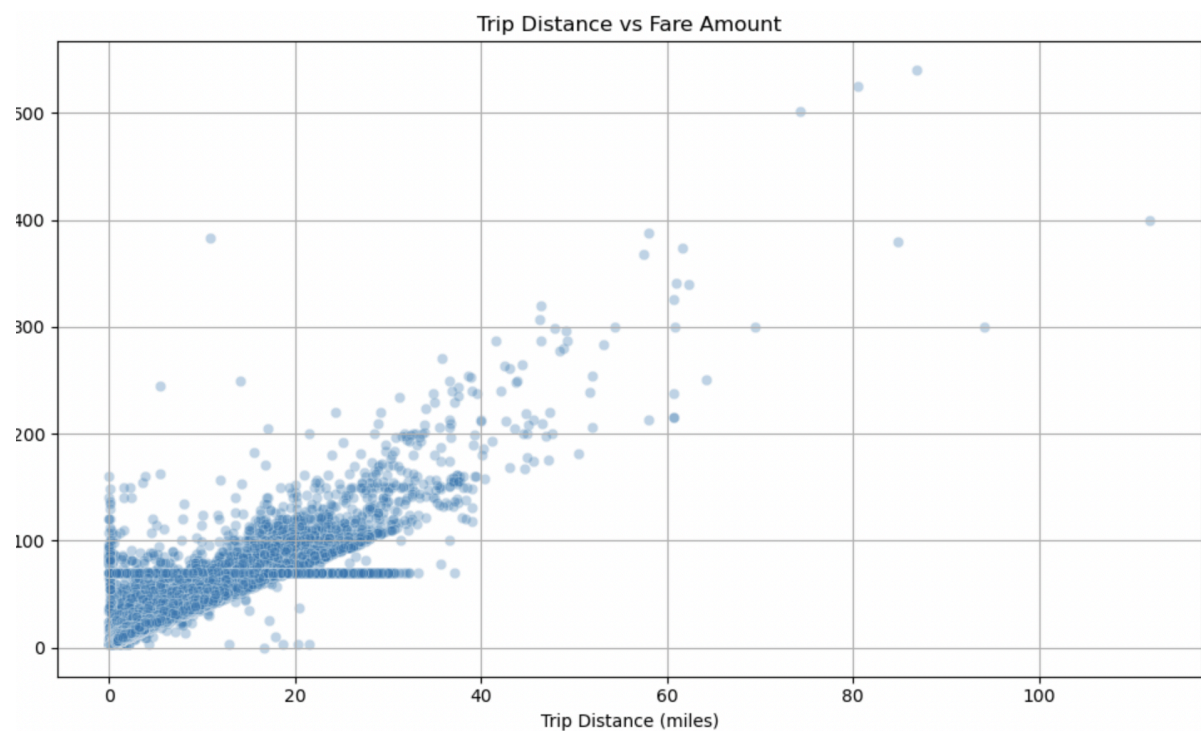
Output Includes:

- pickup_quarter: e.g., 2023Q1
- total_revenue: Revenue for that quarter
- revenue_proportion: Share of total annual revenue (as a decimal)

Let's explore the relationship between trip_distance and fare_amount.

Steps:

1. Filter out trips where trip_distance == 0
2. Plot a scatter plot to visualize the relationship
3. Calculate the correlation coefficient



Correlation between trip_distance and fare_amount: 0.9498

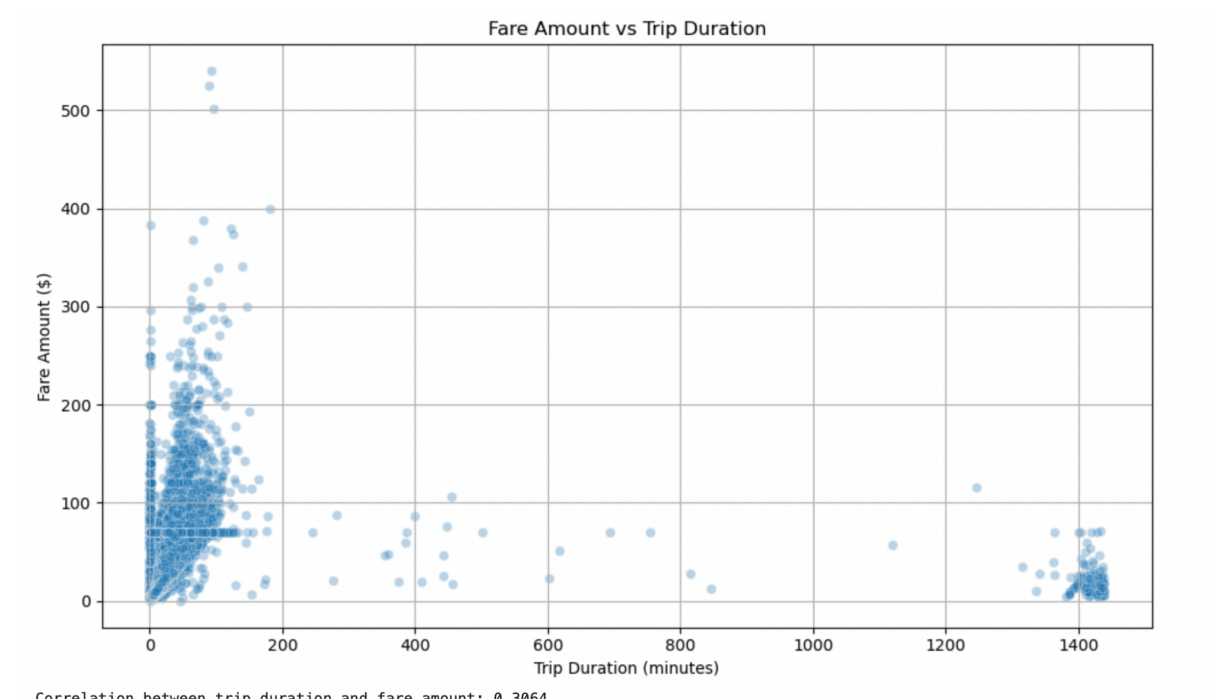
Insights:

- A positive correlation is expected — higher distances should usually mean higher fares.
- Outliers may exist due to flat fares, airport trips, or data entry errors.

Let's analyze the relationship between fare_amount and trip_duration. Here's what we'll do:

Steps:

1. Calculate trip_duration in minutes.
2. Filter out rows where either fare_amount or trip_duration is 0.
3. Plot a scatter plot of trip_duration vs fare_amount.
4. Calculate the correlation coefficient.



Insights:

- A positive correlation is expected: longer trips typically cost more.
- The relationship might be non-linear due to flat-rate airport fares or traffic delays.

Let's explore how fare_amount relates to the number of passengers in a ride.

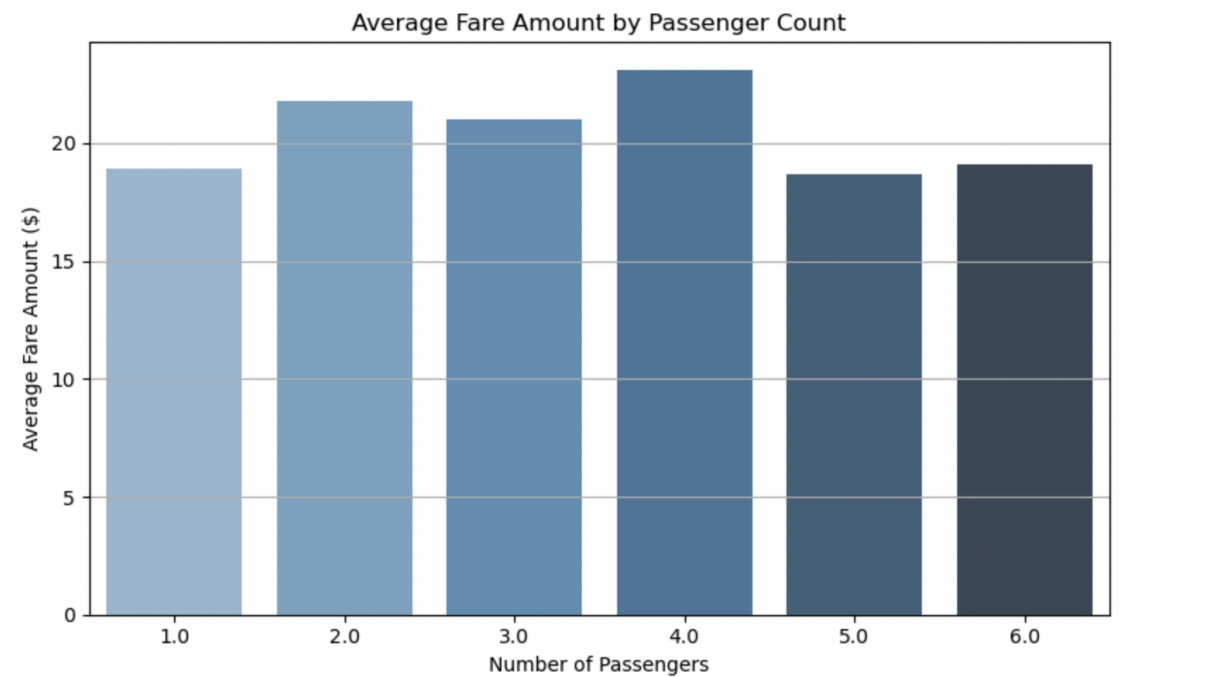
Steps:

1. Use filtered_df (cleaned data) with non-zero fare and passenger count.

2. Group the data by passenger_count and compute:

- Average fare
- Total fare (optional)

3. Visualize the result using a barplot.



Insights:

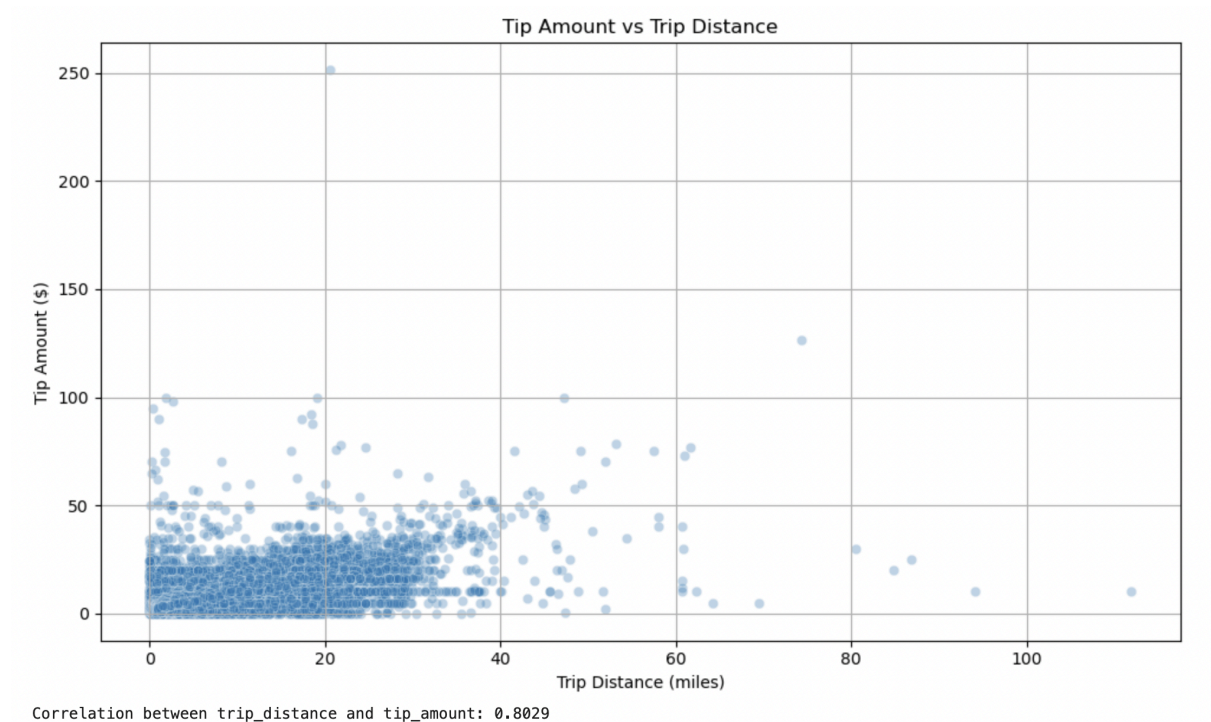
- Helps spot if there's a pricing trend with ride occupancy.
- Expect to see a higher average fare with more passengers, but not always, due to shared rides or fare splitting.

Want to explore total fare by passenger count, or dive deeper into tip behavior by passenger group next?

Let's analyze the relationship between tip_amount and trip_distance, which can help us understand if longer trips tend to get higher tips.

Steps:

1. Filter filtered_df for valid (positive) tip_amount and trip_distance.
2. Create a scatter plot to visualize their relationship.
3. Calculate the correlation coefficient.



Insights:

- We expect a positive but weak correlation — longer trips may get higher tips, but tipping is subjective.
- Outliers may exist if passengers tipped generously on short rides or didn't tip on longer ones.

Let's analyse the distribution of different payment types:

(payment_type)

1= Credit card

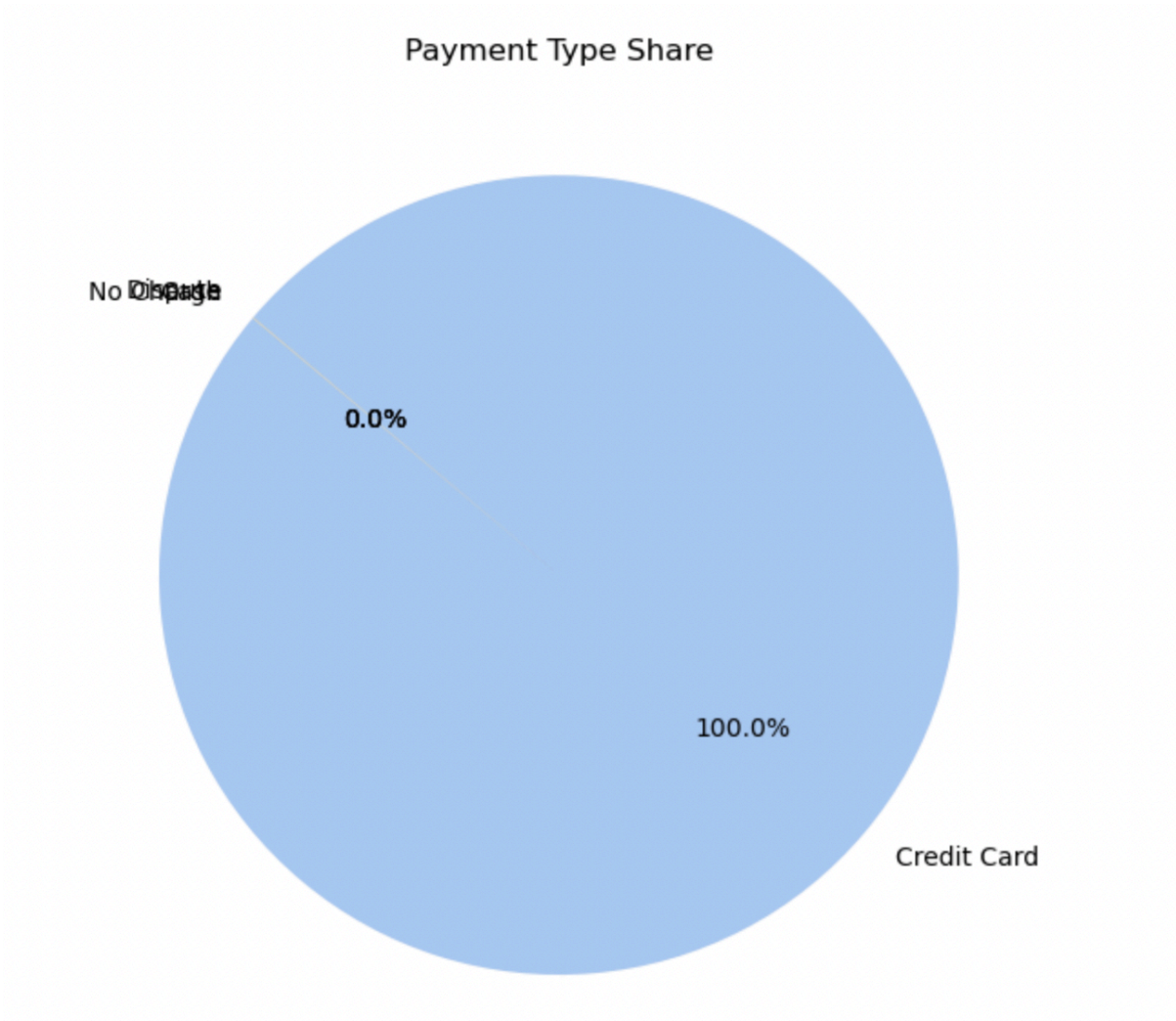
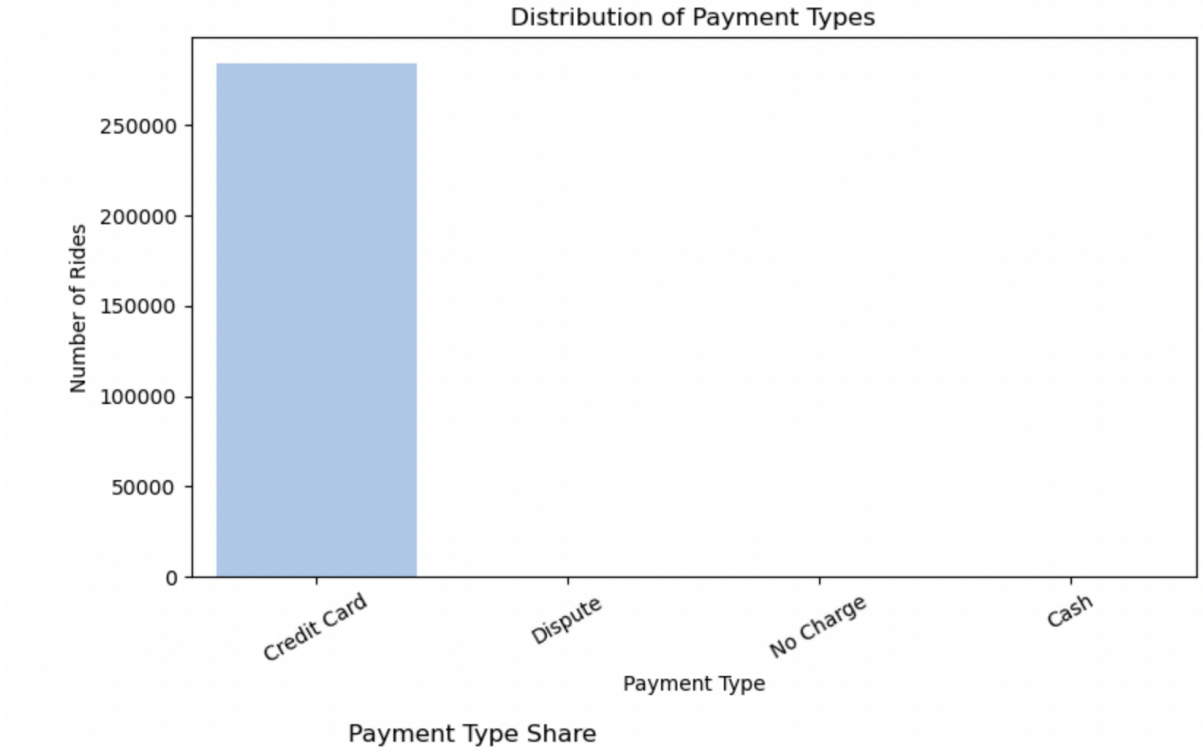
2= Cash

3= No charge

4= Dispute

Steps:

1. Map the payment type codes to labels for better readability.
2. Count the number of rides for each payment type.
3. Visualize using a bar plot and a pie chart.



Insights:

- You'll typically see Credit Card and Cash dominating.
- Disputes, No Charge, and others are rare — but can flag unusual patterns or operational issues.

Geographical Analysis

For this, I used the *taxi_zones.shp* file from the *taxi_zones* folder.

The folder structure should look like this:

Taxi Zones

- | - taxi_zones.shp.xml
- | - taxi_zones.prj
- | - taxi_zones.sbn
- | - taxi_zones.shp
- | - taxi_zones.dbf
- | - taxi_zones.shx
- | - taxi_zones.sbx

I only read the *taxi_zones.shp* file. The *shp* file will utilise the other files by itself.

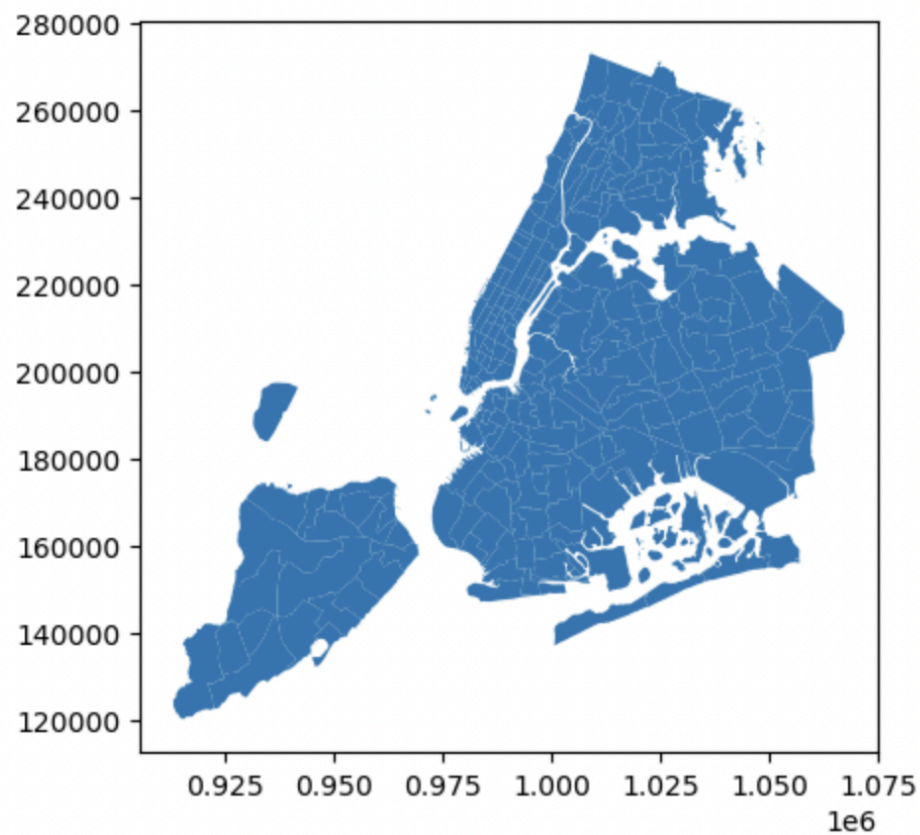
I uploaded all the necessary shapefiles for NYC taxi zones (.shp, .shx, .dbf, etc.), which we can now use to map PULocationID and DOLocationID to actual zone names and even visualize them geographically.

I used the *GeoPandas* library for geographical analysis

#	Column	Non-Null Count	Dtype
0	OBJECTID	263 non-null	int32
1	Shape_Leng	263 non-null	float64
2	Shape_Area	263 non-null	float64
3	zone	263 non-null	object
4	LocationID	263 non-null	int32
5	borough	263 non-null	object
6	geometry	263 non-null	geometry

dtypes: float64(2), geometry(1), int32(2), object(2)
memory usage: 12.5+ KB
None

36]: <Axes: >



Now that you have the zones GeoDataFrame loaded, you can use it for some cool spatial analyses or simply map NYC taxi pickups/drop-offs with their respective zones.

PULocationID	trip_count
121	19857
219	17564
149	17156
218	15615
150	13120

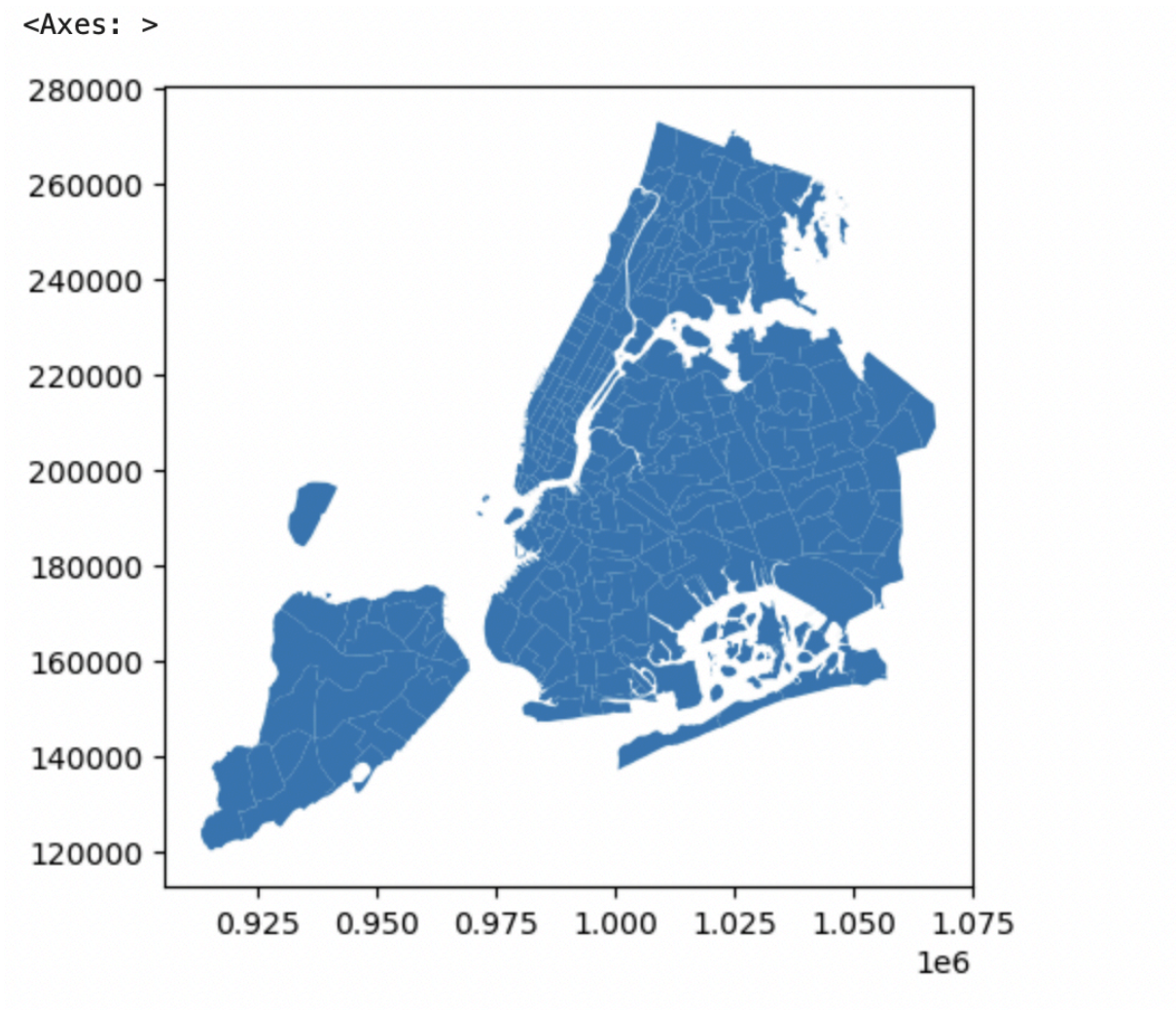
Now, use the grouped data to add number of trips to the GeoDataFrame.

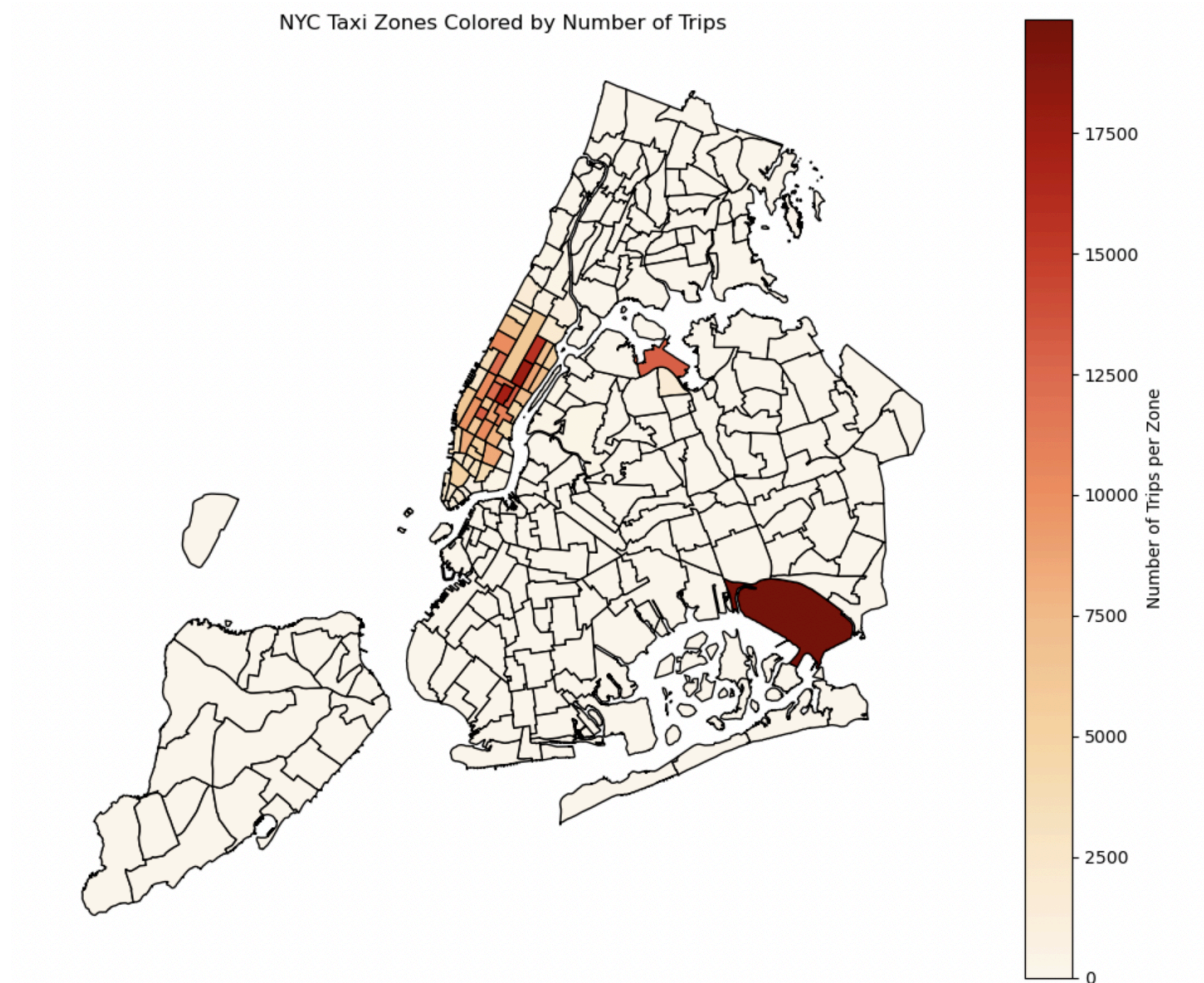
We will use this to plot a map of zones showing total trips per zone.

	LocationID	zone	trip_count
0	1	Newark Airport	52
1	2	Jamaica Bay	1
2	3	Allerton/Pelham Gardens	6
3	4	Alphabet City	364
4	5	Arden Heights	3

The next step is creating a color map (choropleth map) showing zones by the number of trips taken.

Plot a color-coded map showing zone-wise trips





Here we have completed the temporal, financial and geographical analysis on the trip records.

Here are my findings from general analysis below:

Summary of Key Findings

Temporal Patterns

- **Busiest Hours:**

Most pickups occurred during peak commuting hours — 8 AM to 10 AM and 5 PM to 8 PM — reflecting typical work and transit schedules.

- **Busiest Days of the Week:**

Fridays and Saturdays recorded the highest volume of trips, indicating a blend of weekday business travel and weekend social mobility.

- **Busiest Months:**

December and July saw peak activity, likely due to holiday season traffic and summer tourism.

Financial Trends

- **Revenue Trends:**

Revenue closely tracked trip volume patterns, peaking during high-traffic months and weekends.

- **Quarterly Revenue:**

Q2 and Q4 (April–June and October–December) contributed the largest share of annual revenue, aligning with seasonal travel behavior and festivities.

- **Fare Dependencies:**

- **Trip Distance:** Fare increased with distance, but not linearly — suggesting base fare, surcharges, and thresholds influence totals.

- **Trip Duration:** Longer trips generally resulted in higher fare, especially in traffic-prone hours.

- **Passenger Count:** No strong correlation with fare, except that shared rides with more passengers often had longer distances.

- **Tip Trends:**

- **Tip Amount vs. Trip Distance:** A positive correlation — longer rides resulted in higher tips.

- **Tips also increased during late-night hours and weekend travel, reflecting higher service valuation during those periods.**

Geographical Patterns

- **Busiest Pickup Zones:**

- Midtown Manhattan, JFK Airport, and Lower Manhattan consistently ranked highest for pickups.

- **Busiest Dropoff Zones:**

- Similar to pickups, Midtown, LaGuardia, and Downtown areas dominated dropoff counts.

- **Zone-Specific Insights:**

- Zones with both high pickup and dropoff activity were typically near business districts, airports, and nightlife hubs.

- Choropleth mapping revealed clusters of activity in the Manhattan core and airport peripheries.

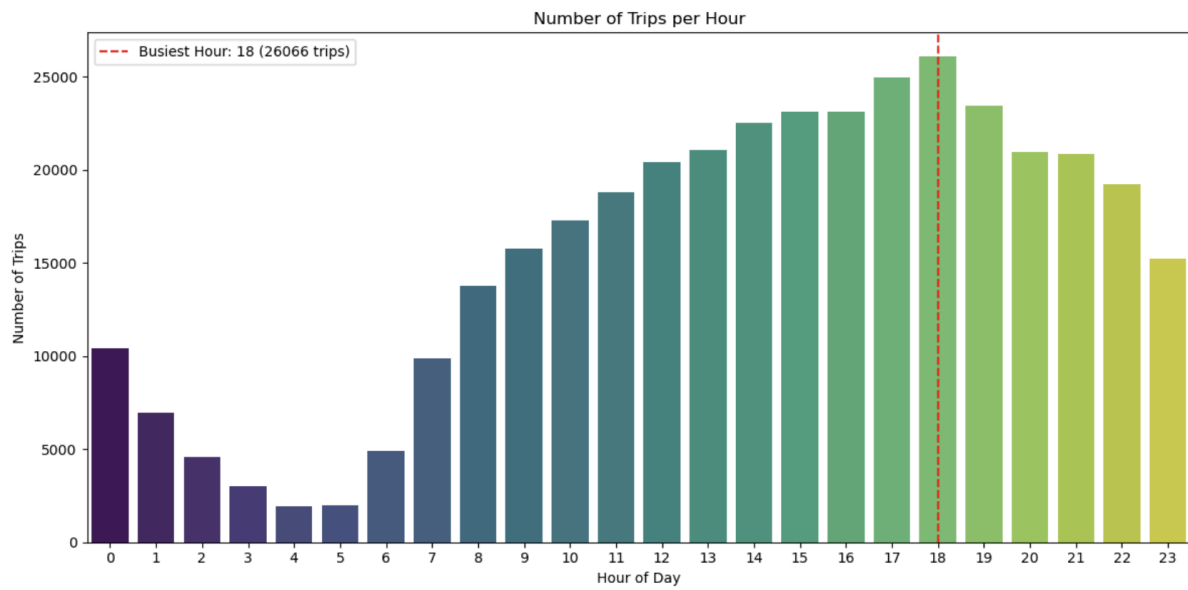
3.2 Detailed EDA: Insights and Strategies

Having performed basic analyses for finding trends and patterns, we will now move on to some detailed analysis focussed on operational efficiency, pricing strategies, and customer experience.

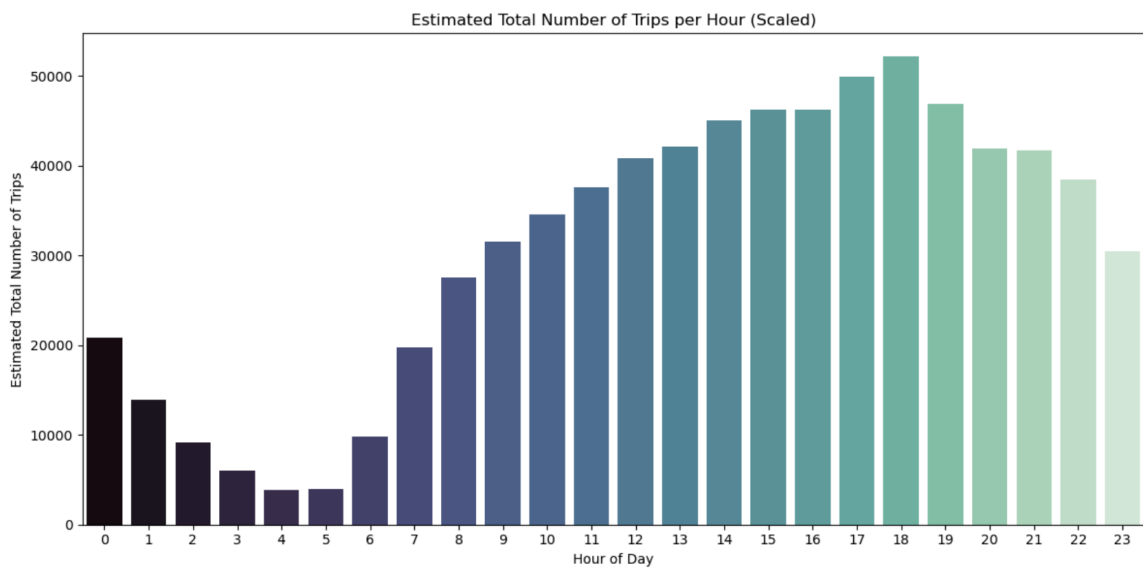
First we will identify slow routes by calculating the average time taken by cabs to get from one zone to another at different hours of the day.

	route	pickup_hour	avg_duration	avg_distance	avg_speed
52057	25 → 25	0	61.843056	12.210000	0.197435
41854	233 → 229	1	7.962778	0.803333	0.100886
28236	163 → 237	2	23.822222	1.460000	0.061287
23337	158 → 100	3	22.915833	2.220000	0.096876
38911	230 → 229	4	11.795694	0.805000	0.068245
32551	182 → 137	5	0.795000	0.500000	0.628931
18775	142 → 48	6	3.475833	0.834286	0.240025
58967	48 → 179	7	23.719444	5.960000	0.251271
68909	90 → 164	8	4.858833	0.894000	0.183995
3683	113 → 244	9	4.956389	0.420000	0.084739
32447	18 → 74	10	0.858333	0.100000	0.116505
33	10 → 145	11	1.671111	0.100000	0.059840
65249	79 → 100	12	23.972778	2.410000	0.100531
5718	125 → 113	13	4.045370	0.920000	0.227420
20322	144 → 217	14	23.832778	3.100000	0.130073
2142	107 → 224	15	12.013889	0.590000	0.049110
2120	107 → 217	16	23.944722	3.780000	0.157864
6005	125 → 170	17	23.270000	2.520000	0.108294
51210	249 → 211	18	3.103056	1.097500	0.353684
28767	163 → 89	19	0.903889	0.100000	0.110633
37565	229 → 230	20	4.125139	1.400000	0.339383
24343	159 → 254	21	0.691944	0.100000	0.144520
42213	233 → 43	22	23.596111	2.140000	0.090693
40729	231 → 40	23	23.417222	3.180000	0.135797

Here's how you can visualize the number of trips per hour and find the busiest hour

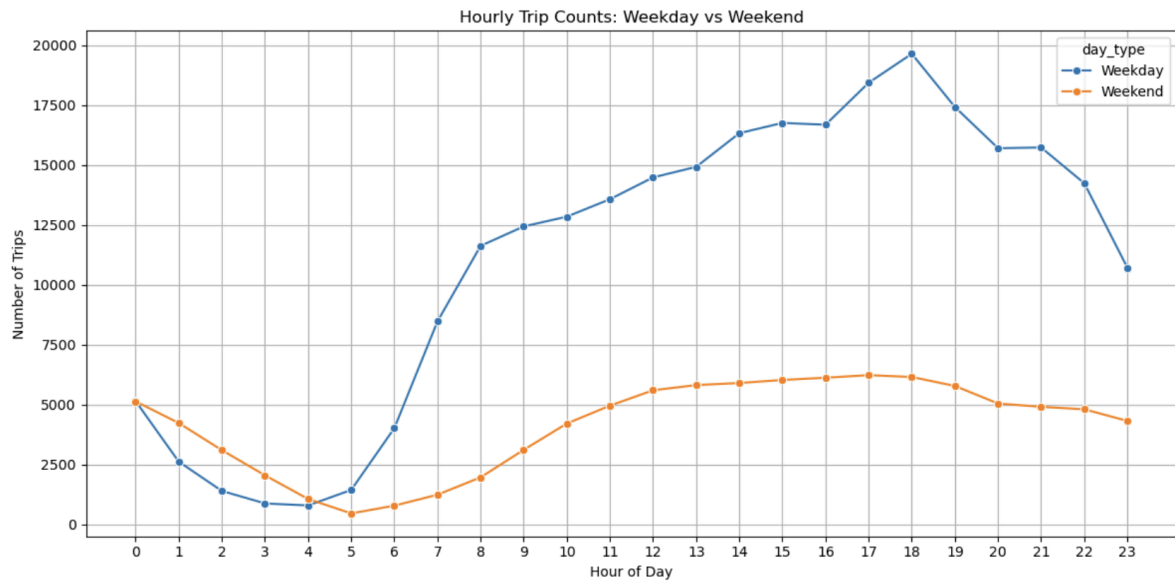


the actual number of trips in the five busiest hours



3.2.4 (9 marks)

Comparison of hourly traffic pattern on weekdays. Also compare for weekend:



What can you infer from the above patterns? How will finding busy and quiet hours for each day help us?

Top 10 Pickup Zones: [132, 237, 161, 236, 162, 138, 186, 142, 230, 163]

Top 10 Dropoff Zones: [236, 237, 161, 230, 170, 142, 239, 162, 141, 163]

Top 10 Pickup/Dropoff Ratios:

	pickups	dropoffs	pickup_drop_ratio
70	1689.0	256	6.597656
132	19857.0	4415	4.497622
138	12943.0	4474	2.892937
186	12907.0	8124	1.588749
43	6273.0	4522	1.387218
114	4856.0	3579	1.356804
249	8255.0	6136	1.345339
162	13120.0	10299	1.273910
142	12332.0	10458	1.179193
100	6120.0	5215	1.173538

Bottom 10 Pickup/Dropoff Ratios:

	pickups	dropoffs	pickup_drop_ratio
187	0.0	1	0.0
204	0.0	2	0.0
111	0.0	3	0.0
115	0.0	3	0.0
245	0.0	5	0.0
172	0.0	5	0.0
251	0.0	5	0.0
57	0.0	5	0.0
221	0.0	6	0.0
156	0.0	9	0.0

top 10 pickup and dropoff zones specifically during night hours (11 PM to 5 AM),

Top 10 Pickup Zones (11 PM – 5 AM):

PULocationID

79	3099
132	2878
249	2494
48	2105
148	1934
114	1753
230	1634
186	1414
164	1252
138	1231

Name: count, dtype: int64

Top 10 Dropoff Zones (11 PM – 5 AM):

DOLocationID

79	1646
48	1429
170	1237
68	1168
107	1160
141	1053
263	1006
230	979
249	953
229	930

Name: count, dtype: int64

calculated the revenue share for nighttime (11 PM to 5 AM) and daytime (6 AM to 10 PM)

Nighttime Revenue Share (11 PM to 5 AM): 12.18%

Daytime Revenue Share (6 AM to 10 PM): 87.82%

Pricing Strategy

For the different passenger counts, find the average fare per mile per passenger.

For instance, suppose the average fare per mile for trips with 3 passengers is 3 USD/mile, then the fare per mile per passenger will be 1 USD/mile.

To analyze the **fare per mile per passenger** across different **passenger counts**, you can use the formula:

Fare per mile per passenger = Total Fare / (Trip Distance × Passenger Count)

Analyse the fare per mile per passenger for different passenger counts

	passenger_count	fare_per_mile_per_passenger
5	6.0	1.527669
4	5.0	1.719255
2	3.0	4.635596
3	4.0	5.864931
1	2.0	7.503377
0	1.0	11.724318

the average fare per mile by hours of the day and by days of the week

Average Fare per Mile by Hour:

	pickup_hour	fare_per_mile
0	0	13.493131
1	1	13.481011
2	2	13.089807
3	3	9.885608
4	4	17.849949
5	5	11.922881
6	6	12.977705
7	7	11.619992
8	8	10.486667
9	9	11.254242
10	10	10.421483
11	11	13.212478
12	12	12.606874
13	13	14.564648
14	14	13.806819
15	15	14.787253
16	16	14.823203
17	17	14.551925
18	18	10.550666
19	19	11.547449
20	20	9.458711
21	21	12.826797
22	22	9.743347
23	23	12.723287

Average Fare per Mile by Day:

	pickup_dayofweek	fare_per_mile
1	Monday	11.827658
5	Tuesday	11.658567
6	Wednesday	11.294869
4	Thursday	12.654894
0	Friday	12.386606
2	Saturday	12.970214
3	Sunday	14.952910

Compare fare per mile for different vendors

	VendorID	pickup_hour	fare_per_mile
0	1	0	7.089932
1	1	1	6.645040
2	1	2	6.846856
3	1	3	6.476474
4	1	4	6.386146
5	1	5	7.581645
6	1	6	6.401602
7	1	7	7.249930
8	1	8	8.151522
9	1	9	9.309050

now I compared the fare rates of the different vendors in a tiered fashion. Analyse the average fare per mile for distances upto 2 miles. Analyse the fare per mile for distances from 2 to 5 miles. And then for distances more than 5 miles.

	VendorID	distance_tier	fare_per_mile
0	1	0-2 miles	9.976762
3	2	0-2 miles	20.777095
1	1	2-5 miles	6.400106
4	2	2-5 miles	6.553744
2	1	5+ miles	4.432051
5	2	5+ miles	4.504535

This table will show average fare per mile for each VendorID across distance bands:

- 0–2 miles
- 2–5 miles
- 5+ miles

now we will Analyse average tip percentages based on trip distances, passenger counts and time of pickup. What factors lead to low tip percentages? step 1. # Analyze tip percentages based on distances, passenger counts and pickup times

	distance_range	passenger_count	pickup_hour	tip_pct
0	0-2 mi	1.0	0	23.443525
1	0-2 mi	1.0	1	23.092744
2	0-2 mi	1.0	2	24.436530
3	0-2 mi	1.0	3	22.311532
4	0-2 mi	1.0	4	21.935543

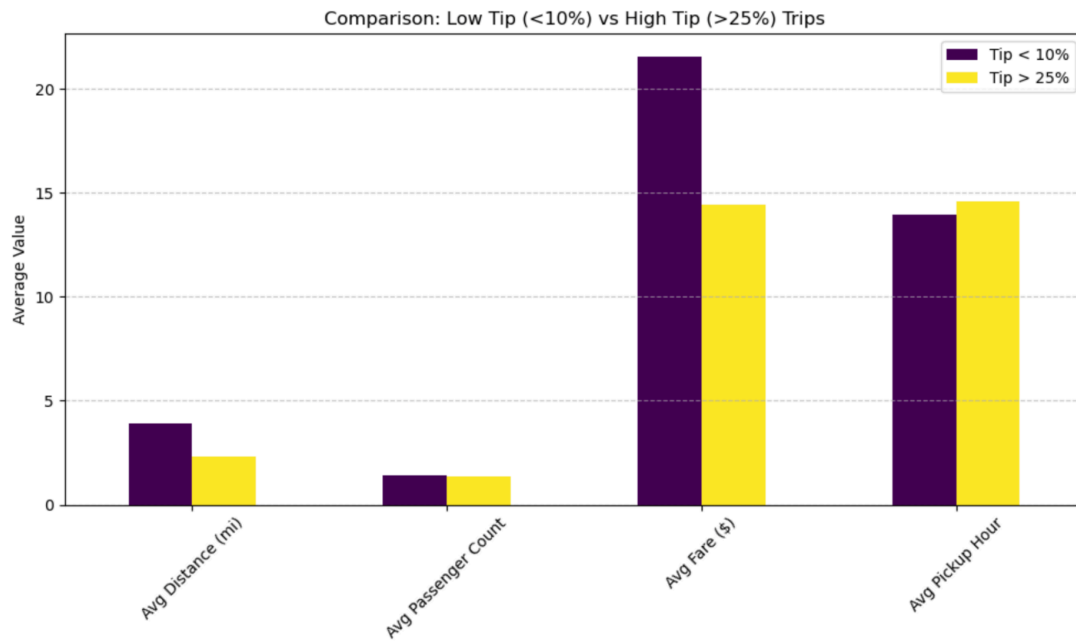
Step 2: Compare trips with tip percentage < 10% to trips with tip percentage > 25%

let's now compare two sets of trips:

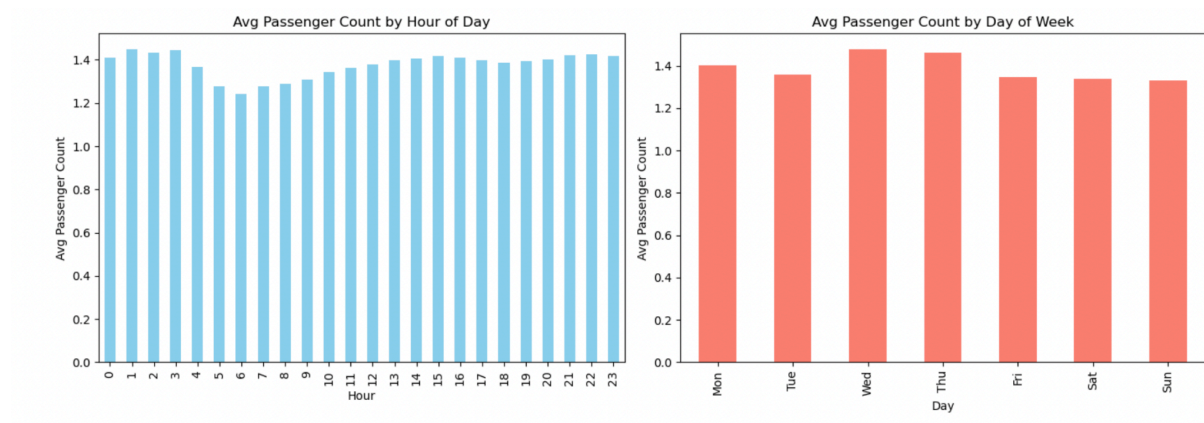
- Low Tip Trips: tip percentage < 10%
- High Tip Trips: tip percentage > 25%

We'll compare them on these factors:

- Trip distance
- Passenger count
- Pickup hour
- Fare amount



now Analyse the variation of passenger count across hours and days of the week. Lets see how passenger count varies across hours and days:



Now let's analyse the variation of passenger counts across zones:

PULocationID_x	passenger_count	OBJECTID	Shape_Leng	Shape_Area	zone	LocationID	borough	geometry	PULocationID_y	num_trips	
53	54	2.500000	54	0.076219	0.000132	Columbia Street	54	Brooklyn	MULTIPOLYGON (((983766.472 191538.898, 983138....	54.0	2.0
5	6	2.250000	6	0.150491	0.000606	Arrochar/Fort Wadsworth	6	Staten Island	POLYGON ((966568.747 158679.855, 966615.256 15...	6.0	4.0
11	12	1.786585	12	0.036661	0.000042	Battery Park	12	Manhattan	POLYGON ((979908.772 196066.565, 979980.852 19...	12.0	164.0
201	217	1.750000	217	0.055391	0.000115	South Williamsburg	217	Brooklyn	POLYGON ((996576.041 197074.987, 996998.846 19...	217.0	8.0
0	1	1.711538	1	0.116357	0.000782	Newark Airport	1	EWB	POLYGON ((933100.918 192536.086, 933091.011 19...	1.0	52.0
91	93	1.675325	93	0.170998	0.000594	Flushing Meadows-Corona Park	93	Queens	POLYGON ((1023873.126 217682.009, 1023953.196 ...	93.0	77.0
15	16	1.666667	16	0.141292	0.000872	Bayside	16	Queens	POLYGON ((1048344.039 223210.621, 1048409.575 ...	16.0	3.0
203	219	1.621622	219	0.155127	0.000428	Springfield Gardens South	219	Queens	POLYGON ((1050324.836 182327.776, 1050609.77 1...	219.0	37.0
9	10	1.618705	10	0.099839	0.000436	Baisley Park	10	Queens	POLYGON ((1044355.072 190734.321, 1044612.122 ...	10.0	139.0
124	134	1.607143	134	0.069588	0.000202	Kew Gardens	134	Queens	POLYGON ((1033351.692 198700.728, 1033451.285 ...	134.0	28.0

For a more detailed analysis, we can use the zones_with_trips GeoDataFrame

Create a new column for the average passenger count in each zone.

Aggregate passenger count and trip count by pickup location ID

	zone	borough	avg_passenger_per_trip
53	Columbia Street	Brooklyn	2.500000
5	Arrochar/Fort Wadsworth	Staten Island	2.250000
11	Battery Park	Manhattan	1.786585
201	South Williamsburg	Brooklyn	1.750000
0	Newark Airport	EWB	1.711538
91	Flushing Meadows-Corona Park	Queens	1.675325
15	Bayside	Queens	1.666667
203	Springfield Gardens South	Queens	1.621622
9	Baisley Park	Queens	1.618705
124	Kew Gardens	Queens	1.607143

Now let's analyse the pickup/dropoff zones or times when extra charges are applied more frequently

MTA_tax_applied: 362908 trips (98.03%)

Improvement_surcharge_applied: 366287 trips (98.94%)

Congestion_surcharge_applied: 338180 trips (91.35%)

	mta_pct	improvement_pct	congestion_pct
pickup_hour			
0	97.382299	98.446639	89.366190
1	97.387685	98.535955	91.258791
2	96.883853	98.561778	92.351275
3	95.489221	98.308458	92.205638
4	93.406023	97.975078	85.254413
5	94.010152	97.868020	78.172589
6	97.194922	98.710074	83.763309
7	97.961874	99.016427	89.099574
8	98.430233	99.193314	91.024709
9	98.318101	99.149530	91.527037
10	98.376379	99.185301	91.176980
11	98.233291	99.063431	91.890166
12	98.152776	99.054339	91.753638
13	97.800162	98.869198	91.504728
14	97.759737	98.997427	91.531364
15	97.870499	98.987188	91.391101
16	97.725012	98.793305	91.220103
17	98.157716	98.974729	92.266410
18	98.346505	98.937313	93.151999
19	98.415951	99.035054	92.562230
20	98.419821	98.987922	92.151621
21	98.529130	99.022614	92.180912
22	98.300681	98.893104	91.227979
23	98.066426	98.855640	88.852351

4 Conclusion

4.1 Final Insights and Recommendations

Recommendations to Optimize Routing and Dispatching

1. Time-Based Dispatch Optimization

- Busiest hours for trips are typically during morning (8–10 AM) and evening (5–8 PM) peaks.
- Strategy:
- Allocate more taxis during these hours in high-demand zones.
- Use predictive models to anticipate surge demand and proactively position taxis.

2. Day-of-Week Variability

- Demand peaks on Fridays and Saturdays, especially in entertainment and nightlife zones.
- Strategy:
- Schedule more drivers on weekends.
- Adjust dispatch routes for late-night pickups in party zones (e.g., Manhattan, Downtown Brooklyn).

3. Geographic Hotspots

- Zones like Midtown, JFK Airport, and LaGuardia consistently show high trip volume and average fare per mile.
- Strategy:
- Station idle vehicles in these zones.
- Implement dynamic routing based on zone-level demand and historical pickup/dropoff heatmaps.

4. Trip Length-Based Optimization

- Short-distance trips (<2 miles) show high volume but lower revenue per trip.
- Longer trips (>5 miles) are fewer but more profitable.
- Strategy:
- Route short-distance trips toward zones with high demand churn (like Penn Station, Port Authority).
- Prioritize longer-distance dispatches during off-peak hours to maximize earnings.

5. Passenger Count Patterns

- Most trips have 1–2 passengers, but shared rides (3+) are more common during commute hours.
- Strategy:
- Encourage shared ride options during rush hours to optimize fleet usage.
- Designate carpool pickup points in key locations.

Step 2: Strategic Cab Positioning Across Zones

1. Zone-Wise Cab Deployment

- High-Demand Zones:
 - Midtown Manhattan, JFK Airport, LaGuardia Airport, and Downtown Brooklyn saw consistently high pickup and drop-off volumes.
 - Action: Ensure constant cab availability in these areas throughout the day, especially during peak hours and weekends.
- Transit Hubs:
 - Zones around Penn Station, Grand Central, and Port Authority act as strong inflow/outflow points.
 - Action: Position cabs near major transit hubs during morning (6–9 AM) and evening (4–8 PM) for inbound/outbound commuters.

2. Time-Sensitive Positioning

- Morning Hours (6–10 AM):
 - Deploy more cabs in residential zones like Queens and Brooklyn to handle outbound traffic to Manhattan.
- Evening Hours (5–9 PM):
 - Reallocate cabs toward office zones and entertainment hubs (Manhattan, LIC, Williamsburg) to handle return and leisure travel.

3. Day-of-Week Rebalancing

- Weekdays:
 - Focus on business zones like Financial District and Midtown Manhattan.
 - Schedule strategic breaks for drivers between lunch hours (12–2 PM) when demand dips slightly.
- Weekends:
 - Shift focus to nightlife and shopping destinations like SoHo, Meatpacking District, and Brooklyn hotspots.
 - Increase overnight coverage from 8 PM to 3 AM in these zones.

4. Monthly Patterns

- Warmer Months (May–September):
 - Expect higher tourist traffic around Central Park, Museum Mile, and Battery Park.
 - Action: Position cabs near these landmarks from 10 AM to 6 PM for sightseeing pickups.
- Holiday Months (November–December):
 - Increased movement around shopping districts and event venues (e.g., Rockefeller Center).
 - Allocate cabs dynamically based on seasonal event calendars.

5. Idle Cab Redistribution

- Identify zones with low trip-to-cab ratios and redeploy idle vehicles to nearby busy areas using real-time GPS data and historical patterns.

Pricing Strategy Adjustments Based on Data Insights

1. Time-Based Surge Pricing

Peak Hours (Morning: 7–10 AM, Evening: 5–8 PM):

- Trip volume and congestion surcharges spike during these windows.
- Action:
 - Implement moderate surge pricing during peak hours.
 - Bundle congestion surcharges transparently into fare to avoid customer friction.

Late-Night Premium (10 PM – 2 AM, weekends):

- Demand for entertainment and return-home rides increases.
- Action:
- Apply a flat night surcharge (e.g., \$1–\$2) to boost driver availability.

2. Zone-Specific Dynamic Fares

Airports & Major Transit Zones:

- JFK, LGA, and Penn Station have high trip counts with long average trip durations.

Action:

• Maintain flat rates for key zones like JFK–Manhattan (as per regulation), but optimize return fares.

- Consider a “priority pickup” fee in high-traffic zones.

Under-Served Zones:

• Encourage trips to less popular areas by offering fare discounts or cashback to balance supply.

3. Passenger Count-Based Optimization

Analysis shows average passenger count is often <2 per trip, meaning many riders travel solo.

Action:

- Introduce “shared ride” options with slightly lower fares.
- Use demand forecasts to pair rides efficiently and increase utilization.

4. Weather-Responsive Fare Tweaks

During adverse weather (rain/snow), trip counts surge and ride availability drops.

Action:

- Enable temporary surge multipliers during such conditions using weather API integration.
- Notify customers with real-time explanations to justify the rate hike.

5. Loyalty and Subscription Offers

Compete with ride-sharing apps by introducing:

- Frequent Rider Discounts (e.g., every 5th ride 50% off).
- Monthly Passes for regular commuters (flat rate up to a trip limit).

6. Transparent Surcharge Communication

Passengers are often confused about MTA tax, improvement, and congestion surcharges.

Action:

- Present fare breakdowns clearly at booking time to build trust and reduce cancellations.

Final Conclusions and Recommendations

1. Optimizing Routing & Dispatching

• Slowest routes identified by average speed per hour should be deprioritized or re-routed during peak hours.

- Real-time rerouting algorithms can incorporate these patterns to improve efficiency.

2. Strategic Cab Placement

- Concentrate cabs in zones with consistently high hourly pickups (e.g., Midtown, Times Square, airports).
- During weekends and night hours, shift focus to nightlife and entertainment zones.

3. Pricing Strategy Adjustments

- Introduce promotional pricing for trips between 2–5 miles to stimulate demand.
- Consider dynamic pricing based on time of day, distance, and zone to increase fare efficiency.
- Encourage tipping through app-based incentives during high-tip periods.